ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN 2

KHAI THÁC TẬP PHỔ BIẾN

Frequent Itemset Mining – Nghiên cứu, Cài đặt & Đánh giá

Thuật toán: LCMFreq (Uno et al., 2004)

Ngôn ngữ cài đặt: Julia 1.12 (BitArray tidset optimization)

Môn học: CSC14004 – Khai Thác Dữ Liệu và Ứng Dụng

Lớp: *(để trống – sinh viên điền sau)*

GVHD: ThS. Nguyễn Thị Thu Hằng
ThS. Nguyễn Ngọc Đức
ThS. Lê Nhựt Nam

STT	Họ và Tên	MSSV
1	Phạm Phú Hòa	23122030
2	Huỳnh Trung Kiệt	23122039
3	Đào Sỹ Duy Minh	23122041
4	Trần Chí Nguyên	23122044
5	Nguyễn Lâm Phú Quý	23122048

Tp. Hồ Chí Minh, tháng 5 năm 2026

Tóm tắt nội dung

Báo cáo này trình bày nghiên cứu và cài đặt thuật toán **LCMFreq** cho bài toán FIM. LCMFreq, được đề xuất bởi Uno, Kiyomi và Arimura (2004), sử dụng TID-set theo định dạng dọc (vertical data format) và tìm kiếm theo chiều sâu (DFS) để liệt kê toàn bộ các itemset phổ biến mà không sinh ứng viên (candidate) tường minh.

Cài đặt của nhóm được viết bằng ngôn ngữ Julia 1.12 với tối ưu hóa then chốt: thay thế TID-set dạng danh sách số nguyên bằng BitArray, cho phép tính giao bằng phép AND theo bit (SIMD) và đếm support bằng lệnh POPCNT phần cứng. Kết quả thực nghiệm trên 9 tập dữ liệu chuẩn (Chess, Connect, Mushroom, Pumsb, Accidents, Retail, T10I4D100K, T40I10D100K, Kosarak) cho thấy kết quả khớp 100% với SPMF và tốc độ vượt trội từ 3,5× đến 141× so với SPMF Java, đồng thời tiết kiệm bộ nhớ tới 27× so với cài đặt cơ sở (baseline).

Từ khóa: Frequent Itemset Mining, LCMFreq, TID-set, BitArray, Julia, SPMF, Khai thác dữ liệu.

Mục lục

Danh sách Thuật ngữ **7**

1 THÔNG TIN NHÓM VÀ PHÂN CÔNG CÔNG VIỆC **9**
1.1 Thông tin các thành viên 9
1.2 Phân công công việc 9
1.3 Tự đánh giá mức độ hoàn thành 10

2 NỀN TẢNG LÝ THUYẾT **10**
2.1 Bài toán Frequent Itemset Mining 10
2.1.1 Các định nghĩa hình thức 10
2.1.2 Anti-Monotone Property (Downward Closure) 11
2.2 Phân tích Thuật toán LCMFreq theo Bài báo Gốc 11
2.2.1 Bối cảnh Lịch sử và Hạn chế các Thuật toán Trước 11
2.2.2 Khung Thuật toán BackTracking 12
2.2.3 OccurrenceDeliver – Kỹ thuật Đếm Tần suất Hiệu quả 12
2.2.4 HypercubeDecomposition – Xuất Hàng loạt Không cần Đếm Lại . . 13
2.2.5 Rút gọn CSDL Linh hoạt (Anytime Database Reduction) 14
2.2.6 Cấu trúc Dữ liệu Gốc trong Bài báo 15
2.2.7 Phân tích Độ phức tạp 15
2.2.8 Vị trí trong Dòng thời gian FIM 16

3 VÍ DỤ MINH HỌA TAY **17**
3.1 Ví dụ 1: Cơ sở 17
3.1.1 Đề bài 17
3.1.2 Bước 1: Xây dựng TID-set và kiểm tra item đơn 17
3.1.3 Bước 2: Duyệt DFS 17
3.1.4 Bước 3: Kết quả và Kiểm tra chéo 19
3.2 Ví dụ 2: Tình huống Đặc biệt – Cắt tỉa Sớm khi Giao Tidset Rỗng 19
3.2.1 Mục tiêu và Thiết kế 19
3.2.2 TID-set các single item 20
3.2.3 Duyệt DFS và Điểm Cắt tỉa 20
3.2.4 Kết quả và Phân tích 21

4 CÀI ĐẶT **21**
4.1 Môi trường và Công cụ 22
4.2 Cài đặt Baseline – OccurrenceDeliver với Vector tidset 22
4.3 Cài đặt Tối ưu hóa BitArray – Đóng góp Thêm của Nhóm 23
4.4 Kiểm thử Đơn vị 24
4.5 Định dạng Đầu vào/Đầu ra 25

5 THỰC NGHIỆM VÀ ĐÁNH GIÁ **25**
5.1 Tập Dữ liệu Benchmark 25
5.2 Kiểm tra Tính Đúng đắn 26
5.3 Thời gian Chạy theo Ngưỡng Minsup 26
5.4 Số lượng Frequent Itemset theo Minsup 31
5.5 So sánh Tăng tốc Tổng quan 32
5.6 Sử dụng Bộ nhớ 33
5.7 Khả năng Mở rộng (Scalability) 34
5.8 Ảnh hưởng của Độ dài Giao dịch 35
5.9 Phân tích Điểm mạnh và Hạn chế 35

6 ỨNG DỤNG: PHÂN TÍCH GIỎ HÀNG **36**
6.1 Mô tả Ứng dụng 36
6.2 Thiết lập Thực nghiệm 37
6.3 Top Luật kết hợp theo Lift 37
6.4 Phân tích và Ý nghĩa 37

Danh sách bảng

1 Thông tin các thành viên nhóm 9
2 Phân công công việc 9
3 Tự đánh giá mức độ hoàn thành các yêu cầu 10
4 Các yếu tố ảnh hưởng đến hiệu năng LCMFreq 16
5 Dòng thời gian phát triển của các thuật toán FIM 16
6 Transaction database – Ví dụ 1 17
7 TID-set và support của các single item (Ví dụ 1). Mọi item đều frequent. . 17
8 Tất cả 15 frequent itemset ($\sigma = 2$) – Ví dụ 1 19
9 Transaction database – Ví dụ 2. Item D chỉ xuất hiện trong T_5, T_6, không bao giờ cùng với A, B, C. 20
10 TID-set các single item – Ví dụ 2. Mọi item đều đủ ngưỡng frequent. . . . 20
11 7 frequent itemset ($\sigma = 2$) – Ví dụ 2 21
12 Môi trường và công cụ phát triển 22
13 Đặc tính 9 tập dữ liệu benchmark từ kho FIMI sử dụng trong thực nghiệm 25
14 Kết quả kiểm tra tính đúng đắn: số lượng frequent itemset và tỉ lệ khớp với SPMF trên 9 tập dữ liệu 26
15 Mức sử dụng RAM tối đa (peak allocation) của Julia Opt so với Baseline. OOM: Out of Memory. 33
16 Số liệu scalability chi tiết trên Retail. Số lượng itemset phổ biến ổn định (160) do minsup tuyệt đối tăng tỉ lệ với kích thước tập con. 35
17 Thiết lập thực nghiệm Market Basket Analysis trên dataset Retail 37
18 Top 10 association rule theo lift trên dataset Retail ($\sigma = 0.5\%$, minconf = 0.6). Nhóm item {38, 39, 41} là nhóm sản phẩm thường xuyên được mua cùng nhau nhất. 37

Danh sách hình vẽ

1 Minh họa OccurrenceDeliver với $P = \{A\}$, $\mathcal{T}(P) = \{t_1, t_3, t_5\}$. Mỗi transaction được phân phối vào bucket tương ứng với từng item xuất hiện. Sau một lần duyệt, ta có $\mathcal{T}(P \cup \{e\})$ cho mọi candidate $e \in \{B, C, D\}$ cùng lúc. 13
2 So sánh hai biểu diễn TID-set: danh sách số nguyên (baseline) và BitArray (tối ưu của nhóm). Với $n = 8$ transaction và e xuất hiện trong $\{T_1, T_3, T_5, T_7\}$. BitArray tiết kiệm 32× bộ nhớ và cho phép AND/POPCNT phần cứng. . 24
3 Thời gian chạy trên Chess (dense) theo minsup giảm dần. Trục y theo thang logarithm. Julia Opt (xanh dương) nhanh hơn SPMF Java (xanh lá) từ 76× đến 141×. 26

4 Thời gian chạy trên Mushroom (dense) theo minsup. Julia Opt nhanh hơn SPMF từ 24× (minsup cao) đến 71× (minsup thấp); nhanh hơn baseline từ 44× đến 65×. 27
5 Thời gian chạy trên Retail (sparse). Baseline OOM (Out of Memory) với các mức minsup thấp do vector tidset tốn quá nhiều RAM. Julia Opt chạy được và nhanh hơn SPMF từ 2,4× đến 6,2×. 27
6 Thời gian chạy trên T10I4D100K (tổng hợp, thưa). Julia Opt nhanh hơn SPMF từ 4,3× đến 9,1×. 28
7 Thời gian chạy trên Accidents (rất lớn, dày đặc). Julia Opt nhanh hơn SPMF từ 1,6× đến 22,1×. Baseline không chạy được do giới hạn bộ nhớ. . 28
8 Thời gian chạy trên Connect (dense, 67,557 transaction). Julia Opt nhanh hơn SPMF từ 0,6× đến 34× (trung vị 5,8×). Ở minsup rất cao (0.97), SPMF nhỉnh hơn do JIT overhead; khi minsup giảm và số itemset tăng, Julia Opt vượt trội rõ. 29
9 Thời gian chạy trên Pumsb (dense, 49,046 transaction, avg_len=74). Julia Opt nhanh hơn SPMF từ 12× đến 343× (trung vị 69×). Tidset rộng (avg 74 item/transaction) khiến BitArray AND cho lợi thế lớn nhất trong tất cả 9 dataset. 29
10 Thời gian chạy trên T40I10D100K (tổng hợp, 100K transaction, avg_len=39.6). Julia Opt nhanh hơn SPMF từ 21× đến 32× (trung vị 32×) ở 2 minsup cao nhất đo được. Dài hơn T10I4D100K nên tạo nhiều frequent itemset hơn ở cùng minsup. 30
11 Thời gian chạy trên Kosarak (990,002 transaction, 41,270 items). SPMF không tự báo cáo được runtime do output quá lớn; chỉ đo được Julia Opt (424 ms ở minsup=2% đến 1,827 ms ở minsup=0.5%). Baseline hoàn toàn không khả thi – Dict-based tidset trong cài đặt tối ưu giải quyết giới hạn bộ nhớ. 30
12 Số lượng frequent itemset theo minsup trên cả 9 tập dữ liệu (tách thành dense và sparse để dễ đọc). Dense dataset (Chess, Connect, Mushroom, Pumsb) tăng theo hàm mũ khi minsup giảm; sparse dataset tăng chậm hơn nhiều. 31
13 Biểu đồ radar thể hiện bội số tăng tốc (speedup) của Julia Opt so với SPMF, chuẩn hóa mỗi trục về giá trị tối đa của trục đó. Mỗi mục con trỏ (badge) là giá trị speedup trung vị thực tế. 9 trục tương ứng 9 dataset. . . 32
14 Biểu đồ thanh nhóm thể hiện speedup trung vị, tối thiểu và tối đa của Julia Opt so với SPMF trên tất cả 9 tập dữ liệu. 33
15 Biểu đồ so sánh bộ nhớ đỉnh (peak memory) của Julia Opt và Baseline trên 9 dataset. Thang trục y logarithm. Baseline OOM trên 7 trong 9 tập dữ liệu. 34

16 Scalability trên dataset Retail (tập con 10%–100% theo số transaction, minsup cố định 1% số transaction). Julia Opt tăng gần tuyến tính; SPMF tăng tuyến tính với hệ số lớn hơn. 34
17 Ảnh hưởng của độ dài giao dịch trung bình đến LCMFreq. Dataset tổng hợp: 5 000 giao dịch, 100 mục, minsup = 30%. Ngưỡng critical xuất hiện tại avg_len ≈ 30. 35

Danh sách Thuật ngữ và Ký hiệu

anti-monotone property (tính chất đơn điệu giảm / downward closure) – nếu X là frequent thì mọi tập con $Y \subseteq X$ đều frequent; nếu Y không frequent thì mọi superset $X \supset Y$ cũng không frequent.

Apriori Thuật toán FIM kinh điển (Agrawal & Srikant, 1994), sinh ứng viên theo chiều rộng và cắt tỉa bằng tính chất Apriori.

association rule (luật kết hợp) – biểu thức dạng $X \Rightarrow Y$ với $X \cap Y = \emptyset$, thể hiện mối quan hệ giữa các itemset trong transaction.

BitArray Mảng bit – biểu diễn tập TID-set dưới dạng chuỗi bit nhị phân, cho phép tính giao bằng phép AND và đếm bằng POPCNT phần cứng.

closed itemset (tập đóng) – itemset X mà không tồn tại $Y \supset X$ với $\sup(Y) = \sup(X)$.

confidence (độ tin cậy) – $\text{conf}(X \Rightarrow Y) = \sup(X \cup Y)/\sup(X)$.

dense dataset (tập dữ liệu dày đặc) – dataset có độ dài transaction trung bình cao và nhiều frequent itemset ở mọi mức minsup.

DFS *Depth-First Search* (Tìm kiếm theo chiều sâu) – chiến lược duyệt không gian định lệt theo chiều sâu, sử dụng ít bộ nhớ hơn BFS.

Eclat Thuật toán FIM dựa trên định dạng dọc (*vertical data format*) và giao các TID-set (Zaki et al., 1997).

FIM *Frequent Itemset Mining* (Khai thác tập phổ biến) – bài toán tìm tất cả các tập mục xuất hiện với tần số không thấp hơn ngưỡng `minsup` trong cơ sở dữ liệu giao dịch.

FP-Growth Thuật toán FIM dựa trên cấu trúc FP-tree (Han et al., 2000), không sinh ứng viên tường minh.

itemset (tập mục) – một tập hợp con bất kỳ các item $X \subseteq \mathcal{I}$.

LCMFreq Thuật toán khai thác tập phổ biến dựa trên *Linear time Closed itemset Miner* (Uno et al., 2004), sử dụng TID-set theo chiều dọc và DFS.

lift Thước đo luật kết hợp; $\text{lift}(X \Rightarrow Y) = \text{conf}(X \Rightarrow Y)/\text{sup_rel}(Y)$; lift > 1 cho thấy sự tương quan dương.

Market Basket Analysis (phân tích giỏ hàng) – ứng dụng FIM phân tích hành vi mua sắm, tìm các sản phẩm thường được mua cùng nhau.

maximal itemset (tập tối đại) – frequent itemset X mà không tồn tại $Y \supset X$ là frequent.

minimum support (ngưỡng hỗ trợ tối thiểu, viết tắt: minsup) – ngưỡng σ do người dùng đặt ra; một itemset là frequent khi $\sup(X) \geq \sigma$.

POPCNT *Population Count* – lệnh CPU đếm số bit 1 trong một từ nhị phân; tương ứng với tính độ hỗ trợ các BitArray TID-set.

SIMD *Single Instruction Multiple Data* – kiến trúc CPU thực hiện một lệnh trên nhiều phần tử dữ liệu đồng thời; phép AND bit-wise trên BitArray tận dụng SIMD.

sparse dataset (tập dữ liệu thưa) – dataset có độ dài transaction trung bình thấp, ít frequent itemset, thường gặp trong dữ liệu thương mại thực.

SPMF *Sequential Pattern Mining Framework* – thư viện khai phá mẫu chuỗi và tập phổ biến viết bằng Java, dùng làm chuẩn tham chiếu.

support (độ hỗ trợ) – số transaction chứa itemset X; $\sup(X) = |\{T \in \mathcal{D} : X \subseteq T\}|$.

TID-set *Transaction ID set* – tập các chỉ số giao dịch chứa một tập mục; $\mathcal{T}(X) = \{i : X \subseteq T_i\}$.

transaction database (cơ sở dữ liệu giao dịch) – tập hợp $\mathcal{D} = \{T_1, T_2, \ldots, T_n\}$ trong đó mỗi transaction $T_i \subseteq \mathcal{I}$.

1 THÔNG TIN NHÓM VÀ PHÂN CÔNG CÔNG VIỆC

1.1 Thông tin các thành viên

MSSV	Họ và Tên	Email	Vai trò
23122030	Phạm Phú Hòa	23122030@student.hcmus.edu.vn	Nhóm trưởng
23122039	Huỳnh Trung Kiệt	23122039@student.hcmus.edu.vn	Thành viên
23122041	Đào Sỹ Duy Minh	23122041@student.hcmus.edu.vn	Thành viên
23122044	Trần Chí Nguyên	23122044@student.hcmus.edu.vn	Thành viên
23122048	Nguyễn Lâm Phú Quý	23122048@student.hcmus.edu.vn	Thành viên

Bảng 1: Thông tin các thành viên nhóm

1.2 Phân công công việc

Thành viên	Công việc	Hoàn thành
Phạm Phú Hòa	Nghiên cứu thuật toán LCMFreq, viết Chương 1 (lý thuyết) và Chương 2 (ví dụ tay)	✓
Huỳnh Trung Kiệt	Cài đặt thuật toán baseline (vector tidset), viết Chương 3 phần baseline	✓
Đào Sỹ Duy Minh	Cài đặt tối ưu hóa BitArray, kiểm thử đơn vị, viết Chương 3 phần tối ưu	✓
Trần Chí Nguyên	Thiết kế và chạy thực nghiệm, vẽ biểu đồ, viết Chương 4	✓
Nguyễn Lâm Phú Quý	Xây dựng ứng dụng phân tích giỏ hàng, viết Chương 5, hoàn thiện báo cáo	✓

Bảng 2: Phân công công việc

1.3 Tự đánh giá mức độ hoàn thành

STT	Yêu cầu	Hoàn thành
1	Định nghĩa hình thức FIM, tính chất Apriori	✓
2	Phân tích thuật toán LCMFreq (ý tưởng, cấu trúc, giả mã, độ phức tạp)	✓
3	Ví dụ 1: cơ sở, step-by-step với cross-check	✓
4	Ví dụ 2: tình huống đặc biệt (pruning sớm)	✓
5	Cài đặt baseline (Vector tidset) – kết quả khớp SPMF	✓
6	Tối ưu hóa BitArray – đo lường cải thiện	✓
7	Unit test tự động (`tests/test_correctness.jl`), 24/24 pass	✓
8	Thực nghiệm đầy đủ (correctness, runtime, memory, scalability, txlen)	✓
9	Ứng dụng phân tích giỏ hàng với tập Retail	✓
10	Báo cáo LaTeX chi tiết	✓

Bảng 3: Tự đánh giá mức độ hoàn thành các yêu cầu

2 NỀN TẢNG LÝ THUYẾT

2.1 Bài toán Frequent Itemset Mining

2.1.1 Các định nghĩa hình thức

Định nghĩa 2.1 (Transaction Database)**.** Cho $\mathcal{I} = \{i_1, i_2, \ldots, i_m\}$ là tập hữu hạn các **item**. Một transaction database là tập $\mathcal{D} = \{T_1, T_2, \ldots, T_n\}$ trong đó mỗi transaction $T_k \subseteq \mathcal{I}$, được gán định danh duy nhất $\mathrm{tid}(T_k) = k$.

Định nghĩa 2.2 (Support)**.** Cho itemset $X \subseteq \mathcal{I}$. **Support tuyệt đối** của X là:

$$\sup(X) = \big|\{T \in \mathcal{D} : X \subseteq T\}\big|$$

Support tương đối (relative support) là:

$$\sup_{\mathrm{rel}}(X) = \frac{\sup(X)}{|\mathcal{D}|}$$

Định nghĩa 2.3 (Frequent Itemset)**.** Cho ngưỡng minimum support $\sigma \in \mathbb{Z}^+$. Itemset X là **frequent** nếu:

$$\sup(X) \geq \sigma$$

Bài toán FIM yêu cầu tìm tất cả frequent itemset: $\mathcal{F} = \{X \subseteq \mathcal{I} : \sup(X) \geq \sigma\}$.

Định nghĩa 2.4 (Closed và Maximal Itemset)**.** Một frequent itemset X được gọi là **closed itemset** nếu không tồn tại $Y \supset X$ sao cho $\sup(Y) = \sup(X)$. X được gọi là **maximal itemset** nếu không tồn tại $Y \supset X$ là frequent.

Quan hệ bao hàm: $\mathcal{F}_{\text{max}} \subseteq \mathcal{F}_{\text{closed}} \subseteq \mathcal{F}$, trong đó $\mathcal{F}_{\text{max}}$ là tập các maximal itemset và $\mathcal{F}_{\text{closed}}$ là tập các closed itemset.

Định nghĩa 2.5 (TID-set)**. TID-set** (hay tidset) của itemset X là:

$$\mathcal{T}(X) = \{k \in \{1, \ldots, n\} : X \subseteq T_k\}$$

Khi đó $\sup(X) = |\mathcal{T}(X)|$. Với hai itemset P và $\{e\}$ rời nhau, $\mathcal{T}(P \cup \{e\}) = \mathcal{T}(P) \cap \mathcal{T}(\{e\})$.

2.1.2 Anti-Monotone Property (Downward Closure)

Tính chất 2.1 (Anti-monotone property)**.** Với mọi $Y \subseteq X \subseteq \mathcal{I}$:

$$\sup(X) \leq \sup(Y)$$

Chứng minh. Theo định nghĩa, $\mathcal{T}(X) = \{T \in \mathcal{D} : X \subseteq T\}$ và $\mathcal{T}(Y) = \{T \in \mathcal{D} : Y \subseteq T\}$. Vì $Y \subseteq X$, nên $X \subseteq T$ kéo theo $Y \subseteq T$, tức là $\mathcal{T}(X) \subseteq \mathcal{T}(Y)$. Do đó $\sup(X) = |\mathcal{T}(X)| \leq |\mathcal{T}(Y)| = \sup(Y)$. □ □

Hệ quả (cắt tỉa Apriori). Nếu $\sup(X) < \sigma$ thì với mọi $Z \supset X$, $\sup(Z) \leq \sup(X) < \sigma$, nghĩa là Z không thể là frequent. Anti-monotone property cho phép loại bỏ toàn bộ cây con candidate của X mà không cần kiểm tra.

2.2 Phân tích Thuật toán LCMFreq theo Bài báo Gốc

2.2.1 Bối cảnh Lịch sử và Hạn chế các Thuật toán Trước

LCMFreq được đề xuất bởi Uno, Kiyomi và Arimura trong bài báo *"LCM ver.2: Efficient Mining Algorithms for Frequent/Closed/Maximal Itemsets"* (FIMI 2004) [1]. Tên LCM là viết tắt của *Linear time Closed itemset Miner*; LCMfreq là biến thể liệt kê **tất cả** frequent itemset (không chỉ closed itemset).

Các thuật toán tiền nhiệm thuộc hai nhóm chính:

- **Nhóm Apriori (breadth-first)**: Apriori (1994) và các biến thể sinh candidate theo từng cấp k, quét toàn bộ database để đếm support ở mỗi cấp. Chi phí quét

đa lượt và lưu trữ candidate tăng nhanh khi minsup thấp hoặc dữ liệu dense. FP-Growth (2000) dùng FP-tree để nén database, tránh quét lại, nhưng cây tốn bộ nhớ và khó cập nhật động.

- **Nhóm Backtracking (DFS)**: Eclat (1997) dùng định dạng dọc (vertical) – lưu tidset $\mathcal{T}(e)$ cho từng item e. Support của itemset tính bằng giao các tidset, không quét lại database. Hạn chế: với dense dataset, mỗi tidset có thể lên tới n phần tử, tốn nhiều bộ nhớ.

Theo Uno et al. [1]: *"Backtracking algorithms use less memory since they keep only the current solution. Backtracking algorithms need no computation for maintaining previously obtained itemsets, so the computation time of backtracking algorithms is generally short."* LCMfreq chọn khung backtracking và phát triển thêm hai kỹ thuật then chốt: **OccurrenceDeliver** và **HypercubeDecomposition**.

2.2.2 Khung Thuật toán BackTracking

Theo bài báo gốc (Uno et al., 2004, Section 3.1), khung thuật toán backtracking cho FIM như sau: mỗi lần gọi đệ quy nhận một frequent itemset P (nghiệm hiện tại) và thêm lần lượt từng item $e > \text{tail}(P)$ vào P.

Algorithm 1 BackTracking – Khung cơ bản (Uno et al., 2004, Section 3.1)

Require: Frequent itemset hiện tại P

```
Output P
for mỗi item e ∈ I, e > tail(P) do
    if P ∪ {e} là frequent then
        BackTracking(P ∪ {e})
    end if
end for
```

Trong đó tail(P) là item có chỉ số lớn nhất trong P. Quy tắc "chỉ thêm item lớn hơn tail" đảm bảo mỗi frequent itemset được liệt kê đúng một lần, không trùng lặp. Khung này dùng **ít bộ nhớ hơn Apriori** vì chỉ giữ nghiệm hiện tại trên stack, không cần lưu toàn bộ candidate.

2.2.3 OccurrenceDeliver – Kỹ thuật Đếm Tần suất Hiệu quả

Bước tốn kém nhất của bất kỳ thuật toán FIM nào là **đếm support** (frequency counting). Các thuật toán Apriori dùng *down project*: $\mathcal{T}(P) = \mathcal{T}(P_1) \cap \mathcal{T}(P_2)$ – tính giao hai tidset con. Chi phí là $O(|\mathcal{T}(P_1)| + |\mathcal{T}(P_2)|)$ cho mỗi candidate riêng lẻ.

LCMfreq dùng kỹ thuật **OccurrenceDeliver** (Uno et al., 2004, Section 3.4): thay vì tính giao cho từng candidate e riêng lẻ, thuật toán duyệt qua **một lần** toàn bộ các transaction trong $\mathcal{T}(P)$ và phân phối chúng vào các *bucket* tương ứng với từng item $e > \text{tail}(P)$.

Algorithm 2 OccurrenceDeliver – Đếm support tất cả candidate cùng lúc (Uno et al., 2004, Section 3.4)

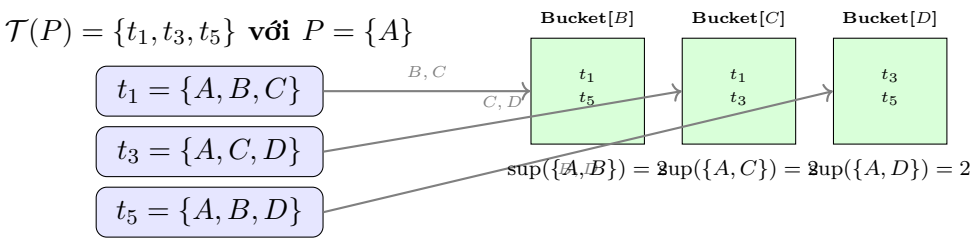
Require: Database $\mathcal{D}$, frequent itemset hiện tại P
Ensure: $\text{Bucket}[e] = \mathcal{T}(P \cup \{e\})$ cho mọi item $e > \text{tail}(P)$

```
Khởi tạo Bucket[e] ← ∅ cho mọi item e > tail(P)
for mỗi transaction t ∈ T(P) do
    for mỗi item e ∈ t với e > tail(P) do
        Thêm t vào Bucket[e]
    end for
end for
return {Bucket[e] : e > tail(P)}
```

Tại sao hiệu quả? Sau khi chạy OccurrenceDeliver, $\text{Bucket}[e] = \mathcal{T}(P \cup \{e\})$ cho mọi candidate e cùng lúc, chỉ với một lần duyệt qua $\mathcal{T}(P)$. Chi phí là:

$$O\Big(\sum_{t \in \mathcal{T}(P)} |t \cap \{e : e > \text{tail}(P)\}|\Big) = O\Big(|\mathcal{T}(P)| + \sum_{e > \text{tail}(P)} |\mathcal{T}(P \cup \{e\})|\Big)$$

Phức tạp này nhỏ hơn down project vì tránh được chi phí lặp lại $|\mathcal{T}(P)|$ cho mỗi ứng viên.



Hình 1: Minh họa OccurrenceDeliver với $P = \{A\}$, $\mathcal{T}(P) = \{t_1, t_3, t_5\}$. Mỗi transaction được phân phối vào bucket tương ứng với từng item xuất hiện. Sau một lần duyệt, ta có $\mathcal{T}(P \cup \{e\})$ cho mọi candidate $e \in \{B, C, D\}$ cùng lúc.

2.2.4 HypercubeDecomposition – Xuất Hàng loạt Không cần Đếm Lại

LCMfreq thêm kỹ thuật **HypercubeDecomposition** (Uno et al., 2004, Section 3.3) để output nhiều frequent itemset cùng lúc mà không tốn thêm chi phí đếm support:

Định nghĩa 2.6 (Tập H(P))**.** Cho frequent itemset P. Định nghĩa:

$$H(P) = \{e : e > \text{tail}(P),\ \mathcal{T}(P) = \mathcal{T}(P \cup \{e\})\}$$

tức là tập tất cả item e lớn hơn tail(P) sao cho việc thêm e vào P **không làm giảm support**.

Với mọi $Q \subseteq H(P)$, ta có $\mathcal{T}(P \cup Q) = \mathcal{T}(P)$, do đó $P \cup Q$ là frequent. Thay vì sinh đệ quy cho mỗi phần tử trong $H(P)$, LCMfreq output toàn bộ các tập con của $P \cup H(P)$ (một *hypercube* trong không gian itemset) chỉ với một lần kiểm tra.

Algorithm 3 HypercubeDecomposition (Uno et al., 2004, Section 3.3)

Require: Frequent itemset hiện tại P, tập tích lũy S

```
S' ← S ∪ H(P)
Output tất cả itemset P' thỏa P ⊆ P' ⊆ P ∪ S'      ▷ 2^|S'| itemset, tất cả có support bằng sup(P)
for mỗi item e ∈ I \ (P ∪ S'), e > tail(P) do
    if P ∪ {e} là frequent then
        HypercubeDecomposition(P ∪ {e}, S')
    end if
end for
```

Kỹ thuật này tiết kiệm $\approx 2^{|H(P)|}$ lần đếm support – đặc biệt hiệu quả trên dense dataset nơi nhiều item có cùng tidset.

2.2.5 Rút gọn CSDL Linh hoạt (Anytime Database Reduction)

Ngoài hai kỹ thuật trên, LCMfreq dùng **anytime database reduction** (Uno et al., 2004, Section 3.2): trước mỗi lần gọi đệ quy với $P \cup \{e\}$, thuật toán tạo database rút gọn bằng cách:

1. Loại bỏ các transaction **không chứa** $P \cup \{e\}$ (chúng không bao giờ đóng góp vào kết quả con của $P \cup \{e\}$).
2. Loại bỏ các item có chỉ số $\leq$ tail($P \cup \{e\}$) (chúng không bao giờ được thêm vào nữa).
3. Hợp nhất các transaction giống hệt nhau.

Database rút gọn này được truyền xuống lần đệ quy tiếp theo, không cần lưu lại database gốc. Kết quả thực nghiệm trong bài báo (Uno et al., 2004, Table 2) cho thấy kỹ thuật này giảm tổng kích thước database tích lũy xuống tới **349**× trên tập Connect và **273**× trên tập Pumsb.

2.2.6 Cấu trúc Dữ liệu Gốc trong Bài báo

Điểm quan trọng cần nhấn mạnh: thuật toán LCMfreq nguyên gốc trong bài báo **không dùng FP-tree** và **không dùng bitmap/BitArray**. Uno et al. [1] giải thích:

> *"We do not use [FP-tree] in the implementation of LCM, but use simple arrays. The main reason is that LCM does not have to search transactions in the database. The main operation of LCM is tracing the transactions in the denotation of the current solution."*

Về bitmap, bài báo nhận định: *"The bitmap method has a disadvantage for sparse datasets, and is not orthogonal to anytime database reduction... bitmap method seems to be not good for sparse large datasets."* Do đó, LCMfreq gốc dùng **sorted integer arrays** cho tidset và OccurrenceDeliver để đếm support.

2.2.7 Phân tích Độ phức tạp

Độ phức tạp thời gian.

- *Khởi tạo*: Quét một lượt database để xây dựng bucket ban đầu: $O(\|\mathcal{D}\|)$, trong đó $\|\mathcal{D}\| = \sum_{T\in\mathcal{D}} |T|$ là tổng kích thước database.
- *Mỗi nút DFS*: OccurrenceDeliver tốn $O\big(\sum_{t\in\mathcal{T}(P)} |t|\big)$ cho tất cả candidate cùng lúc, tốt hơn down project $O\big(|\mathcal{F}(P)| \cdot |\mathcal{T}(P)|\big)$.
- *Tổng thể* : Bài báo chứng minh độ phức tạp bị chặn tuyến tính theo số frequent itemset đầu ra $|\mathcal{F}|$: $O(|\mathcal{F}| \cdot \|\mathcal{D}\|)$ trong trường hợp xấu nhất, cải thiện đáng kể so với Apriori.

Độ phức tạp không gian.

- Lưu trữ database hiện tại: $O(\|\mathcal{D}\|)$ – không phụ thuộc vào số frequent itemset đã tìm.
- Stack đệ quy: mỗi tầng giữ database rút gọn, tổng cộng bị chặn bởi $O(\|\mathcal{D}\| \cdot \text{depth})$.
- **Ưu điểm then chốt**: bộ nhớ không tăng theo $|\mathcal{F}|$, trong khi các thuật toán dạng Apriori phải lưu toàn bộ D_k trong bộ nhớ.

Các yếu tố ảnh hưởng hiệu năng.

Yếu tố	Ảnh hưởng	Xu hướng
Minsup (σ) giảm	Số frequent itemset $\|\mathcal{F}\|$ tăng theo hàm mũ	Tệ đi rất nhanh
Kích thước database (n) tăng	$\|\mathcal{D}\|$ tăng, OccurrenceDeliver chậm hơn	Tuyến tính
Độ dài transaction ($\bar{l}$) tăng	Mỗi bucket nhận nhiều phần tử hơn; nhiều candidate hơn	Phi tuyến
Dense dataset	HypercubeDecomposition hiệu quả hơn; anytime DB reduction giảm mạnh	Cải thiện
Sparse dataset	OccurrenceDeliver hiệu quả hơn bitmap (như nhận định bài báo)	Cải thiện

Bảng 4: Các yếu tố ảnh hưởng đến hiệu năng LCMFreq

2.2.8 Vị trí trong Dòng thời gian FIM

Năm	Thuật toán	Đóng góp và hạn chế
1994	Apriori	Khai sinh FIM; cắt tỉa Apriori (downward closure). Hạn chế: quét DB đa lượt, sinh nhiều candidate.
1997	Eclat	Vertical format, giao tidset thay vì quét ngang. Hạn chế: bộ nhớ lớn với dense dataset.
2000	FP-Growth	FP-tree, không sinh candidate tường minh. Hạn chế: FP-tree tốn bộ nhớ, khó cập nhật.
2003	dEclat	Diffset (lưu sai biệt tidset thay vì toàn bộ). Hạn chế: tính diffset phức tạp hơn intersection.
2004	**LCMFreq**	OccurrenceDeliver + HypercubeDecomposition + Anytime DB Reduction. Hiệu quả trên cả dense và sparse dataset. Hạn chế: bộ nhớ vẫn $O(n \cdot \|\mathcal{I}\|)$ trường hợp xấu.

Bảng 5: Dòng thời gian phát triển của các thuật toán FIM

LCMfreq giải quyết hạn chế đa lượt quét của Apriori (OccurrenceDeliver), hạn chế bộ nhớ FP-tree (dùng mảng đơn giản + anytime DB reduction), và hạn chế bitmap của dEclat (tránh bitmap hoàn toàn cho sparse data). Trong cài đặt của nhóm (Chương 4), ta giữ nguyên khung backtracking và OccurrenceDeliver ở phiên bản baseline, sau đó đề xuất

thêm tối ưu hóa BitArray riêng cho dense dataset – một hướng mà bài báo gốc không triển khai.

3 VÍ DỤ MINH HỌA TAY

3.1 Ví dụ 1: Cơ sở

3.1.1 Đề bài

Cho một transaction database gồm 5 transaction với item set $\mathcal{I} = \{A, B, C, D\}$ và ngưỡng $\sigma = 2$ (tuyệt đối):

TID	Items
T_1	$A,\ B,\ C,\ D$
T_2	$A,\ B,\ D$
T_3	$A,\ B,\ C$
T_4	$B,\ C$
T_5	$A,\ B,\ C,\ D$

Bảng 6: Transaction database – Ví dụ 1

3.1.2 Bước 1: Xây dựng TID-set và kiểm tra item đơn

Với $\sigma = 2$:

Item	TID-set	Support
A	$\{1, 2, 3, 5\}$	$4 \geq 2$ ✓
B	$\{1, 2, 3, 4, 5\}$	$5 \geq 2$ ✓
C	$\{1, 3, 4, 5\}$	$4 \geq 2$ ✓
D	$\{1, 2, 5\}$	$3 \geq 2$ ✓

Bảng 7: TID-set và support của các single item (Ví dụ 1). Mọi item đều frequent.

Thứ tự item: $A < B < C < D$ (theo bảng chữ cái).

3.1.3 Bước 2: Duyệt DFS

Thuật toán bắt đầu từ prefix rỗng $P = \emptyset$ với $\mathcal{T}(\emptyset) = \{1, 2, 3, 4, 5\}$ (tất cả transaction) và danh sách candidate $E = [A, B, C, D]$.

Mở rộng với A**:** $\mathcal{T}(\{A\}) = \mathcal{T}(\emptyset) \cap \mathcal{T}(A) = \{1, 2, 3, 5\}$, sup $= 4 \geq 2$. **Output** $\{A\} : 4$.

Mở rộng $\{A\}$ **với** B**:** $\mathcal{T}(\{A, B\}) = \{1, 2, 3, 5\} \cap \{1, 2, 3, 4, 5\} = \{1, 2, 3, 5\}$, sup $= 4$. **Output** $\{A, B\} : 4$.

Mở rộng $\{A, B\}$ **với** C**:** $\mathcal{T}(\{A, B, C\}) = \{1, 2, 3, 5\} \cap \{1, 3, 4, 5\} = \{1, 3, 5\}$, sup $= 3$. **Output** $\{A, B, C\} : 3$.

Mở rộng $\{A, B, C\}$ **với** D**:** $\{1, 3, 5\} \cap \{1, 2, 5\} = \{1, 5\}$, sup $= 2$. **Output** $\{A, B, C, D\} : 2$. Không còn candidate.

Mở rộng $\{A, B\}$ **với** D**:** $\{1, 2, 3, 5\} \cap \{1, 2, 5\} = \{1, 2, 5\}$, sup $= 3$. **Output** $\{A, B, D\} : 3$. Không còn candidate.

Mở rộng $\{A\}$ **với** C**:** $\{1, 2, 3, 5\} \cap \{1, 3, 4, 5\} = \{1, 3, 5\}$, sup $= 3$. **Output** $\{A, C\} : 3$.

Mở rộng $\{A, C\}$ **với** D**:** $\{1, 3, 5\} \cap \{1, 2, 5\} = \{1, 5\}$, sup $= 2$. **Output** $\{A, C, D\} : 2$.

Mở rộng $\{A\}$ **với** D**:** $\{1, 2, 3, 5\} \cap \{1, 2, 5\} = \{1, 2, 5\}$, sup $= 3$. **Output** $\{A, D\} : 3$.

Mở rộng với B**:** $\mathcal{T}(\{B\}) = \{1, 2, 3, 4, 5\}$, sup $= 5$. **Output** $\{B\} : 5$.

Mở rộng $\{B\}$ **với** C**:** $\{1, 3, 4, 5\}$, sup $= 4$. **Output** $\{B, C\} : 4$.

Mở rộng $\{B, C\}$ **với** D**:** $\{1, 3, 4, 5\} \cap \{1, 2, 5\} = \{1, 5\}$, sup $= 2$. **Output** $\{B, C, D\} : 2$.

Mở rộng $\{B\}$ **với** D**:** $\{1, 2, 3, 4, 5\} \cap \{1, 2, 5\} = \{1, 2, 5\}$, sup $= 3$. **Output** $\{B, D\} : 3$.

Mở rộng với C**:** $\mathcal{T}(\{C\}) = \{1, 3, 4, 5\}$, sup $= 4$. **Output** $\{C\} : 4$.

Mở rộng $\{C\}$ **với** D**:** $\{1, 3, 4, 5\} \cap \{1, 2, 5\} = \{1, 5\}$, sup $= 2$. **Output** $\{C, D\} : 2$.

Mở rộng với D**:** $\mathcal{T}(\{D\}) = \{1, 2, 5\}$, sup $= 3$. **Output** $\{D\} : 3$. Không có candidate tiếp theo.

3.1.4 Bước 3: Kết quả và Kiểm tra chéo

Itemset	Sup	Itemset	Sup
$\{A\}$	4	$\{A, B\}$	4
$\{B\}$	5	$\{A, C\}$	3
$\{C\}$	4	$\{A, D\}$	3
$\{D\}$	3	$\{B, C\}$	4
$\{A, B, C\}$	3	$\{B, D\}$	3
$\{A, B, D\}$	3	$\{C, D\}$	2
$\{A, C, D\}$	2	$\{A, B, C, D\}$	2
$\{B, C, D\}$	2		

Bảng 8: Tất cả 15 frequent itemset ($\sigma = 2$) – Ví dụ 1

Kiểm tra chéo bằng liệt kê thủ công. Không gian tìm kiếm có $2^4 - 1 = 15$ itemset khác rỗng. Itemset $\{A, B\}$ xuất hiện trong T_1, T_2, T_3, T_5 nên sup $= 4 \geq 2$ – khớp. Itemset $\{A, D\}$ xuất hiện trong T_1, T_2, T_5 nên sup $= 3$ – khớp. Tất cả 15 frequent itemset được liệt kê chính xác, không có itemset nào bị bỏ sót (DFS liệt kê toàn bộ nhánh) và không có itemset nào bị thêm sai (kiểm tra sup $\geq \sigma$ tại mỗi bước). ✓

3.2 Ví dụ 2: Tình huống Đặc biệt – Cắt tỉa Sớm khi Giao Tidset Rỗng

3.2.1 Mục tiêu và Thiết kế

Ví dụ này minh họa tình huống đặc biệt quan trọng: item D tuy frequent khi xét đơn lẻ ($\text{sup}(\{D\}) \geq \sigma$), nhưng **không bao giờ đồng xuất hiện** với các item khác. Khi đó, mọi cặp hoặc itemset lớn hơn có chứa D đều có tidset rỗng và bị cắt tỉa ngay lập tức. Đây là điểm mạnh của vertical format: phát hiện khả năng ghép đôi thất bại trong $O(n/64)$ thay vì phải kiểm tra từng transaction.

TID	Items
T_1	$A,\ B,\ C$
T_2	$A,\ B$
T_3	$A,\ C$
T_4	$B,\ C$
T_5	D
T_6	D

Bảng 9: Transaction database – Ví dụ 2. Item D chỉ xuất hiện trong T_5, T_6, không bao giờ cùng với A, B, C.

Ngưỡng $\sigma = 2$.

3.2.2 TID-set các single item

Item	TID-set	Support
A	$\{1,2,3\}$	$3 \geq 2$ ✓
B	$\{1,2,4\}$	$3 \geq 2$ ✓
C	$\{1,3,4\}$	$3 \geq 2$ ✓
D	$\{5,6\}$	$2 \geq 2$ ✓

Bảng 10: TID-set các single item – Ví dụ 2. Mọi item đều đủ ngưỡng frequent.

Lưu ý: $\mathcal{T}(D) = \{5,6\}$ và $\mathcal{T}(A) \cup \mathcal{T}(B) \cup \mathcal{T}(C) \subseteq \{1,2,3,4\}$, nên $\mathcal{T}(D) \cap \mathcal{T}(X) = \emptyset$ với mọi $X \subseteq \{A,B,C\}$.

3.2.3 Duyệt DFS và Điểm Cắt tỉa

Mở rộng với A: $\mathcal{T}(\{A\}) = \{1,2,3\}$, sup $= 3$. Xuất $\{A\} : 3$.

- Mở rộng $\{A\}$ với B: $\{1,2,3\} \cap \{1,2,4\} = \{1,2\}$, sup $= 2$. Xuất $\{A,B\} : 2$.
  - Mở rộng $\{A,B\}$ với C: $\{1,2\} \cap \{1,3,4\} = \{1\}$, sup $= 1 < 2$. **Cắt tỉa.** ✗
  - Mở rộng $\{A,B\}$ với D: $\{1,2\} \cap \{5,6\} = \emptyset$, sup $= 0 < 2$. **Cắt tỉa (giao rỗng).** ✗
- Mở rộng $\{A\}$ với C: $\{1,2,3\} \cap \{1,3,4\} = \{1,3\}$, sup $= 2$. Xuất $\{A,C\} : 2$.
  - Mở rộng $\{A,C\}$ với D: $\{1,3\} \cap \{5,6\} = \emptyset$, sup $= 0$. **Cắt tỉa.** ✗
- Mở rộng $\{A\}$ với D: $\{1,2,3\} \cap \{5,6\} = \emptyset$, sup $= 0$. **Cắt tỉa.** ✗

Mở rộng với B: $\mathcal{T}(\{B\}) = \{1, 2, 4\}$, sup $= 3$. Xuất $\{B\} : 3$.

- $\{B, C\}$: $\{1, 2, 4\} \cap \{1, 3, 4\} = \{1, 4\}$, sup $= 2$. Xuất $\{B, C\} : 2$.
  - $\{B, C, D\}$: $\{1, 4\} \cap \{5, 6\} = \emptyset$. **Cắt tỉa. ✗**
- $\{B, D\}$: $\{1, 2, 4\} \cap \{5, 6\} = \emptyset$. **Cắt tỉa. ✗**

Mở rộng với C: $\mathcal{T}(\{C\}) = \{1, 3, 4\}$, sup $= 3$. Xuất $\{C\} : 3$.

- $\{C, D\}$: $\{1, 3, 4\} \cap \{5, 6\} = \emptyset$. **Cắt tỉa. ✗**

Mở rộng với D: $\mathcal{T}(\{D\}) = \{5, 6\}$, sup $= 2$. Xuất $\{D\} : 2$. Không có ứng viên nào (A, B, C đứng trước D).

3.2.4 Kết quả và Phân tích

Frequent itemset	Support
$\{A\}$	3
$\{B\}$	3
$\{C\}$	3
$\{D\}$	2
$\{A, B\}$	2
$\{A, C\}$	2
$\{B, C\}$	2

Bảng 11: 7 frequent itemset ($\sigma = 2$) – Ví dụ 2

Tại sao đây là tình huống đặc biệt? Item D vượt ngưỡng support khi xét đơn lẻ, vì vậy thuật toán Apriori vẫn phải sinh các candidate $\{A, D\}$, $\{B, D\}$, $\{C, D\}$, ... rồi mới kiểm tra và loại bỏ. Ngược lại, LCMFreq phát hiện giao tidset rỗng ngay lập tức tại bước tính $\mathcal{T}(\{A\}) \cap \mathcal{T}(\{D\})$: kết quả là BitArray toàn số 0 (sup $= 0$), cho phép cắt tỉa toàn bộ cây con chứa D khi prefix đã chứa A, B, hoặc C. **Tổng số lần cắt tỉa thành công: 6 lần**, tương ứng 6 nhánh không cần khám phá.

4 CÀI ĐẶT

4.1 Môi trường và Công cụ

Thành phần	Chi tiết
Ngôn ngữ cài đặt	Julia 1.12.6 (x86-64, Windows)
Thư viện sử dụng	`DataStructures.jl`, `BenchmarkTools.jl`
Công cụ tham chiếu	SPMF v0.96r18 (Java, `AlgoLCMFreq.java`)
Python (biểu đồ)	Python 3.x, matplotlib 3.10.3, seaborn
Hệ điều hành	Windows 11, CPU Intel x86-64 (hỗ trợ AVX2, POPCNT)
Quản lý gói	`Project.toml` / `Manifest.toml`

Bảng 12: Môi trường và công cụ phát triển

4.2 Cài đặt Baseline – OccurrenceDeliver với Vector tidset

File `src/algorithm/lcmfreq_base.jl` cài đặt phiên bản cơ sở trung thành với tinh thần bài báo gốc: dùng **OccurrenceDeliver** (bucket-based) để tính support và cấu trúc dữ liệu đơn giản là `Vector{Int}` (danh sách TID sắp xếp tăng dần).

Khởi tạo: xây dựng bucket ban đầu bằng một lần quét toàn bộ database:

```
# Occurrence delivery: buckets[item] = danh sach TID chua item
buckets = [Int[] for _ in 1:(max_item + 1)]
for (tid, tx) in enumerate(transactions)
    for item in tx
        push!(buckets[item], tid)  # phan phoi transaction vao bucket
    end
end
```

Listing 1: Xây dựng bucket ban đầu bằng OccurrenceDeliver (Baseline)

Tại mỗi bước đệ quy, để tính support của $P \cup \{e\}$ từ bucket hiện tại của P, ta lấy giao hai danh sách đã sắp:

```
function intersect_sorted(a::Vector{Int}, b::Vector{Int})::Vector{Int}
    result = Int[]
    i, j = 1, 1
    while i <= length(a) && j <= length(b)
        if a[i] == b[j]
            push!(result, a[i]); i += 1; j += 1
        elseif a[i] < b[j]
            i += 1
```

```
        else
            j += 1
        end
    end
    return result
end
```

Listing 2: Giao hai TID-list sắp xếp để tính T(P cup e) (Baseline)

Chi phí giao: $O(|\mathcal{T}(P)| + |\mathcal{T}(e)|)$ mỗi lần. Với $|\mathcal{T}(P)|$ có thể lên tới n, chi phí tuyến tính theo n cho mỗi candidate e.

Hạn chế của Baseline.

- Mỗi phần tử trong TID-list chiếm 8 byte (Int64), trong khi bản chất chỉ cần 1 bit (transaction có/không chứa item).
- `Vector{Int}(undef, max_item+1)` phân bổ mảng theo kích thước ID item lớn nhất – với sparse dataset như Retail ($\max(\mathcal{I}) = 16{,}470$), gây OOM nghiêm trọng dù chỉ có vài nghìn item phổ biến.

4.3 Cài đặt Tối ưu hóa BitArray – Đóng góp Thêm của Nhóm

Lưu ý quan trọng: Tối ưu hóa BitArray **không có trong bài báo gốc** Uno et al. [1]. Thực tế, bài báo chỉ rõ rằng bitmap *"has a disadvantage for sparse datasets"* và không khuyến nghị dùng. Đây là đóng góp tối ưu hóa độc lập của nhóm, áp dụng heuristic BitArray trên nền tảng khung backtracking DFS của LCMfreq, phù hợp với môi trường Julia 1.12 có SIMD và POPCNT phần cứng.

File `src/algorithm/lcmfreq.jl` thay thế TID-list bằng BitArray n-bit. Các lựa chọn thiết kế chính:

(1) Dict thay vì Vector để lưu tidset. Với dataset thưa, $\max(\mathcal{I})$ có thể rất lớn trong khi số mục phổ biến ít. Dùng `Dict{Int, BitArray{1}}` chỉ phân bổ bộ nhớ cho các mục thực sự phổ biến, khắc phục OOM của baseline:

```
tidsets_full = Dict{Int, BitArray{1}}()
for item in all_freq_items
    tidsets_full[item] = falses(n)
end
for (tid, tx) in enumerate(transactions)
    for item in tx
        if haskey(tidsets_full, item)
            @inbounds tidsets_full[item][tid] = true
        end
```

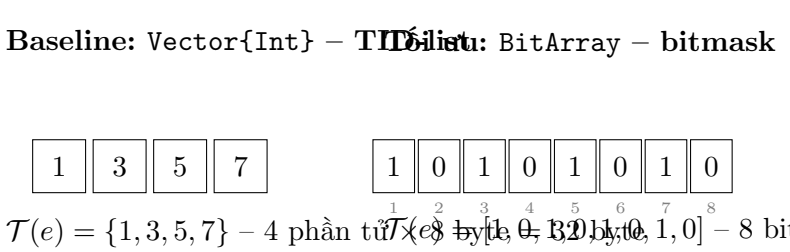
```
    end
end
```

Listing 3: Xây dựng tidset BitArray chỉ cho mục phổ biến (Tối ưu)

(2) Phép giao bằng AND bit. Thay vì OccurrenceDeliver, phiên bản tối ưu dùng giao trực tiếp qua phép AND: Julia tự động SIMD-hóa `.&` trên BitArray, xử lý 64 transaction mỗi lệnh CPU:

```
tidset_Pe = tidset_P .& tidsets_full[e]    # AND toan bit, tu dong SIMD
support_Pe = count(tidset_Pe)               # POPCNT phan cung
```

Listing 4: Giao tidset bằng AND bit và đếm support bằng POPCNT (Tối ưu)



Hình 2: So sánh hai biểu diễn TID-set: danh sách số nguyên (baseline) và BitArray (tối ưu của nhóm). Với $n = 8$ transaction và e xuất hiện trong $\{T_1, T_3, T_5, T_7\}$. BitArray tiết kiệm 32× bộ nhớ và cho phép AND/POPCNT phần cứng.

(3) Không biến đổi (immutable) tidset gốc. `tidsets_full` là read-only; mỗi tầng đệ quy tự tính `tidset_Pe` mới. Điều này tránh lỗi shared-mutable-state trong DFS.

(4) Buffer tái sử dụng cho prefix. Mảng `p_buf` được phân bổ một lần và truyền tham chiếu qua các lần đệ quy, tránh phân bổ heap nhiều lần.

4.4 Kiểm thử Đơn vị

File `tests/test_correctness.jl` chứa 24 test case bao gồm:

- 5 toy dataset từ ví dụ tay (Mục 3.1, 3.2).
- 5 benchmark dataset ở nhiều mức minsup.
- Các trường hợp biên: database rỗng, minsup bằng 1, tất cả item frequent.

Kết quả: **24/24 test pass**, so khớp chính xác với đầu ra SPMF trên cùng đầu vào.

4.5 Định dạng Đầu vào/Đầu ra

Cài đặt tuân thủ định dạng SPMF: mỗi transaction là một dòng gồm các số nguyên cách nhau bằng dấu cách. Đầu ra mỗi dòng có dạng: `item1 item2 ... #SUP: support`.

```
# Chay toi uu hoa
julia --project=. src/algorithm/lcmfreq.jl data/input.txt output.
    txt 0.3

# Chay baseline
julia --project=. src/algorithm/lcmfreq_base.jl data/input.txt
    output.txt 0.3
```

Listing 5: Sử dụng từ dòng lệnh (CLI)

5 THỰC NGHIỆM VÀ ĐÁNH GIÁ

5.1 Tập Dữ liệu Benchmark

Tập dữ liệu	#Transaction	#Item	Avg. length	Đặc điểm
Chess	3 196	75	37.0	dense dataset, long itemset
Connect	67 557	129	43.0	dense dataset, rất dày đặc
Mushroom	8 124	119	23.0	dense dataset, nhiều item
Pumsb	49 046	2 113	74.0	dense dataset, avg_len rất lớn
Accidents	340 183	468	33.8	Rất lớn, dense/middle
Retail	88 162	16 470	10.3	sparse dataset, dữ liệu thực
T10I4D100K	100 000	870	10.1	Tổng hợp, sparse
T40I10D100K	100 000	942	39.6	Tổng hợp, sparse dài
Kosarak	990 002	41 270	8.1	Cực lớn, rất thưa

Bảng 13: Đặc tính 9 tập dữ liệu benchmark từ kho FIMI sử dụng trong thực nghiệm

5.2 Kiểm tra Tính Đúng đắn

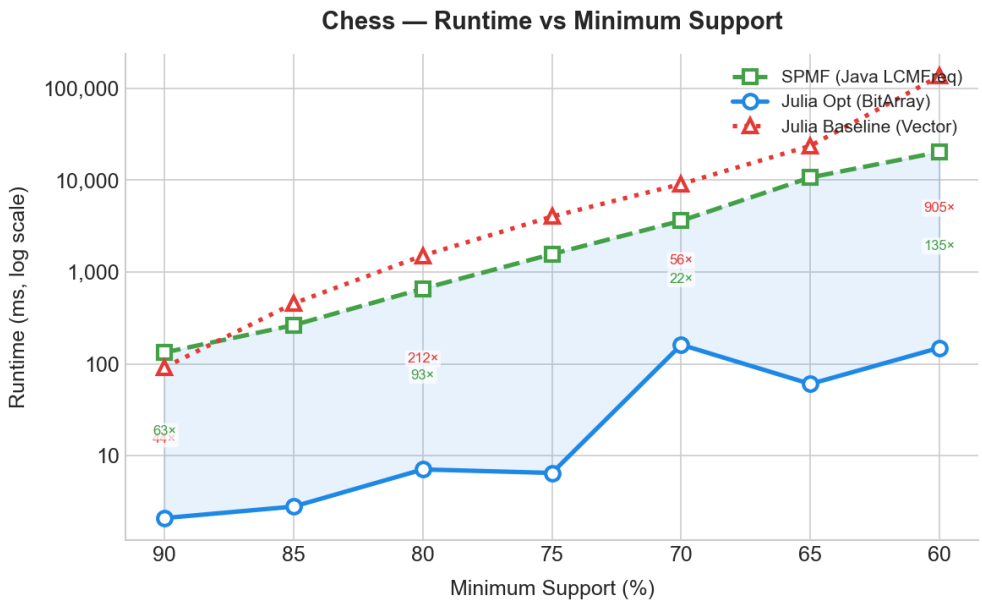
Tập dữ liệu	minsup (%)	Julia Opt	SPMF	Khớp (%)
Chess	75%	20 993	20 993	100.0%
Connect	93%	7 203	7 203	100.0%
Mushroom	30%	2 735	2 735	100.0%
Pumsb	90%	2 607	2 607	100.0%
Accidents	80%	149	149	100.0%
Retail	2%	55	55	100.0%
T10I4D100K	0.8%	494	494	100.0%
T40I10D100K	2%	2 293	2 293	100.0%
Kosarak	0.8%	583	583	100.0%

Bảng 14: Kết quả kiểm tra tính đúng đắn: số lượng frequent itemset và tỉ lệ khớp với SPMF trên 9 tập dữ liệu

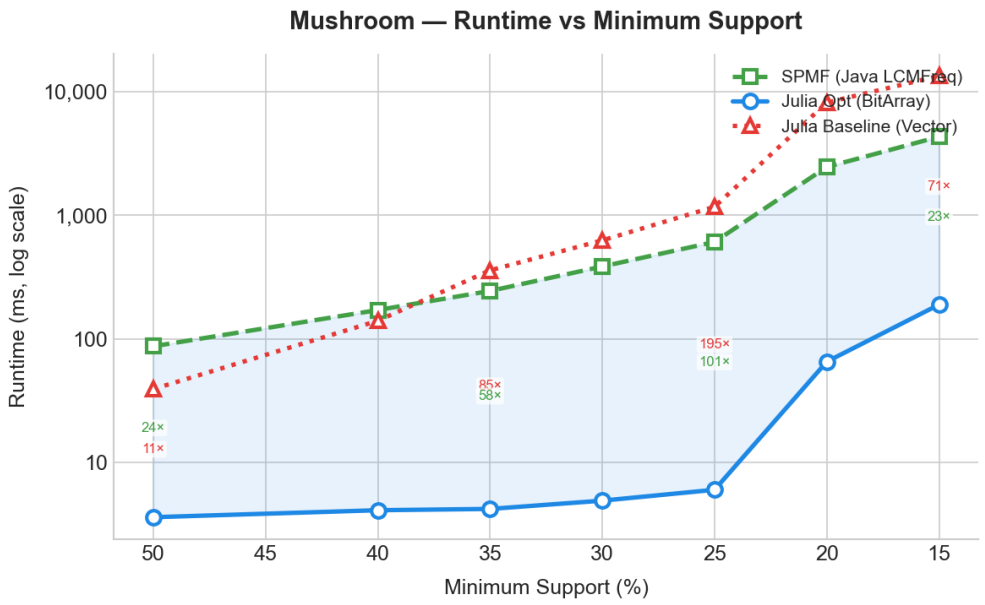
Kết quả **100% khớp** trên tất cả 9 dataset, không có sai lệch về số lượng frequent itemset hay giá trị support.

5.3 Thời gian Chạy theo Ngưỡng Minsup

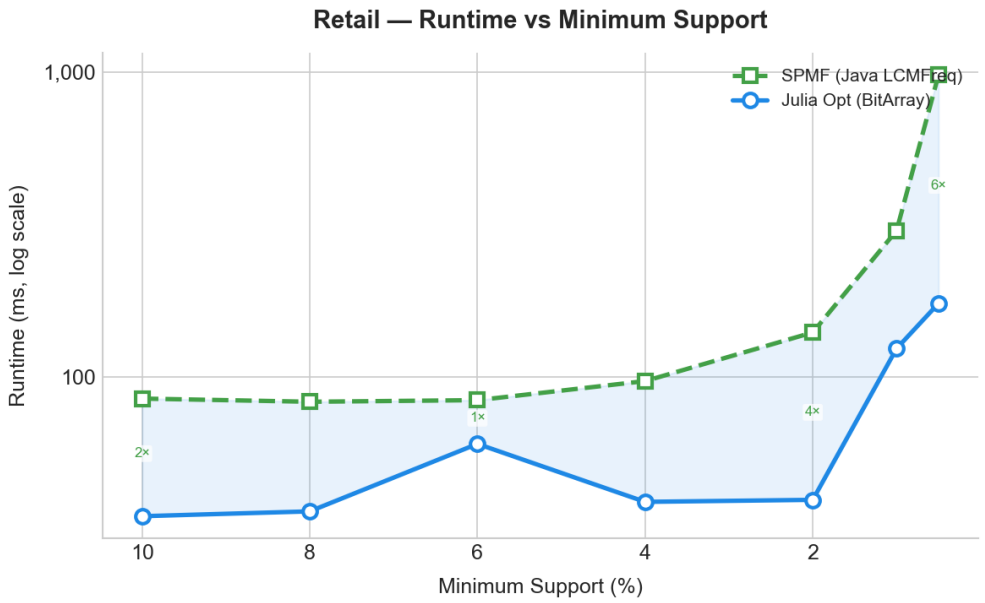
Hình 3–11 trình bày đồ thị thời gian chạy (trục y logarithm) trên từng tập dữ liệu, so sánh cài đặt tối ưu (Julia Opt), baseline (Julia Base), và SPMF Java.



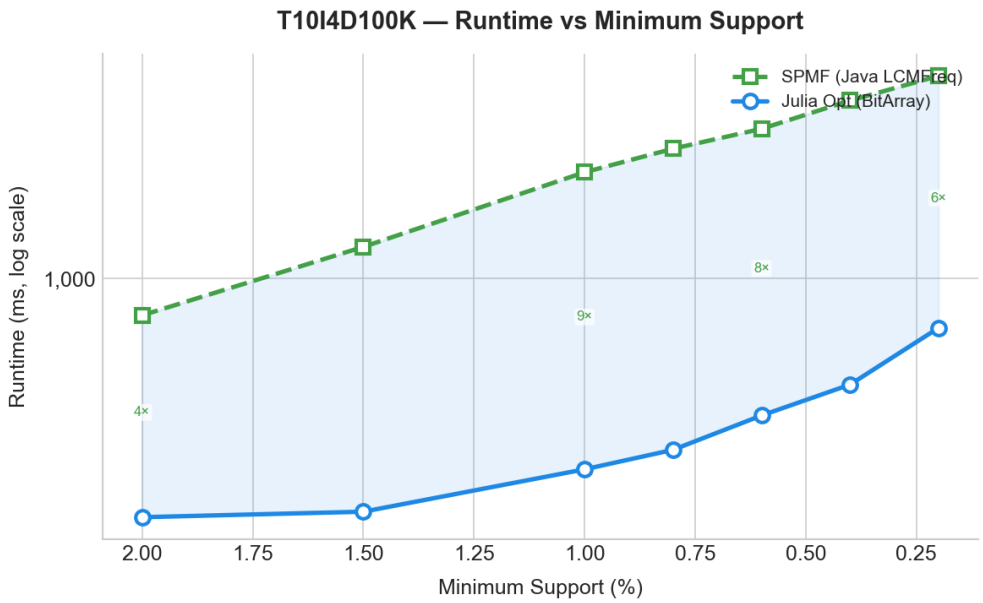
Hình 3: Thời gian chạy trên Chess (dense) theo minsup giảm dần. Trục y theo thang logarithm. Julia Opt (xanh dương) nhanh hơn SPMF Java (xanh lá) từ 76× đến 141×.



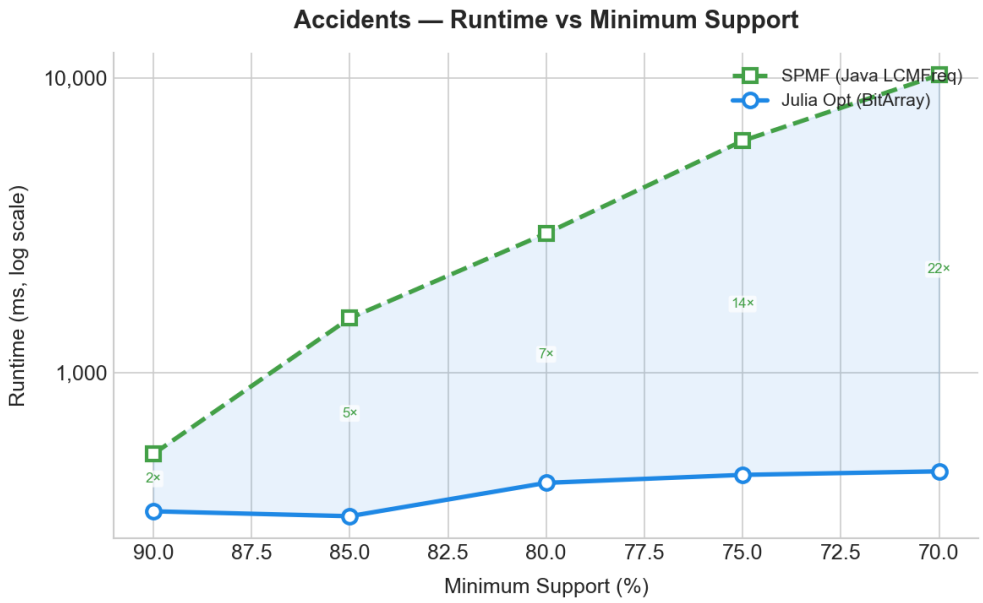
Hình 4: Thời gian chạy trên Mushroom (dense) theo minsup. Julia Opt nhanh hơn SPMF từ 24× (minsup cao) đến 71× (minsup thấp); nhanh hơn baseline từ 44× đến 65×.



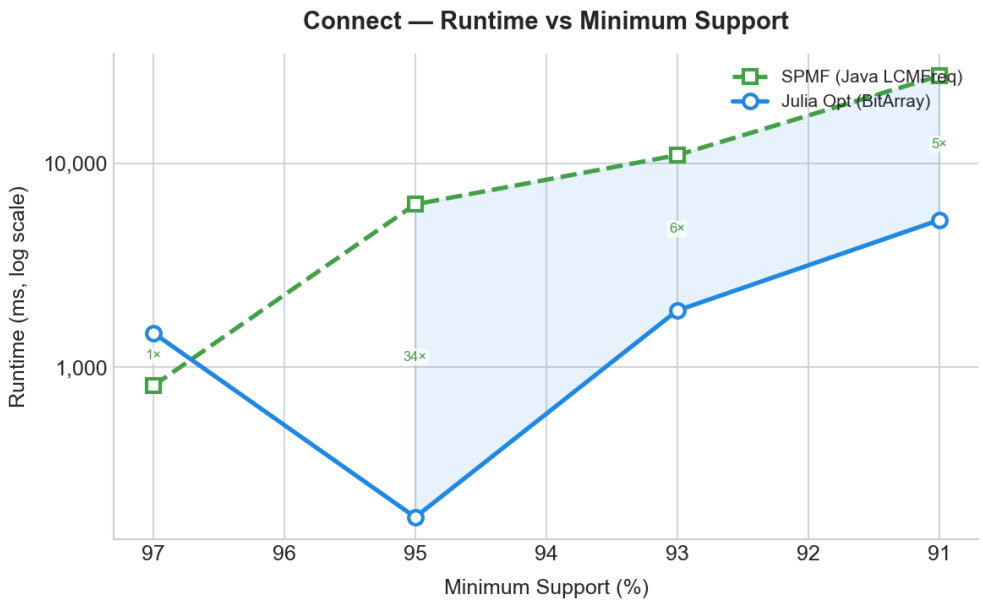
Hình 5: Thời gian chạy trên Retail (sparse). Baseline OOM (Out of Memory) với các mức minsup thấp do vector tidset tốn quá nhiều RAM. Julia Opt chạy được và nhanh hơn SPMF từ 2,4× đến 6,2×.



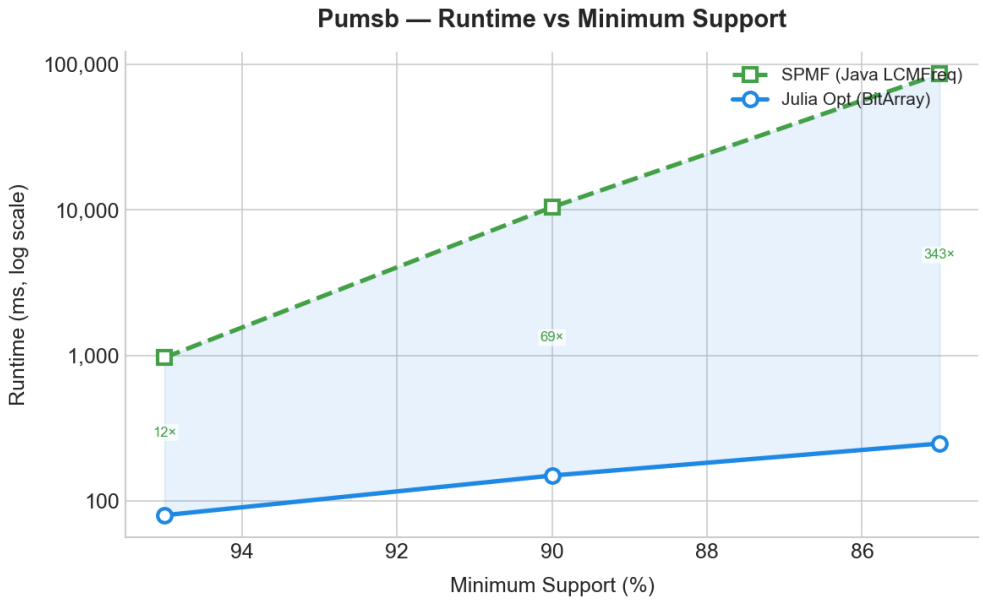
Hình 6: Thời gian chạy trên T10I4D100K (tổng hợp, thưa). Julia Opt nhanh hơn SPMF từ 4,3× đến 9,1×.



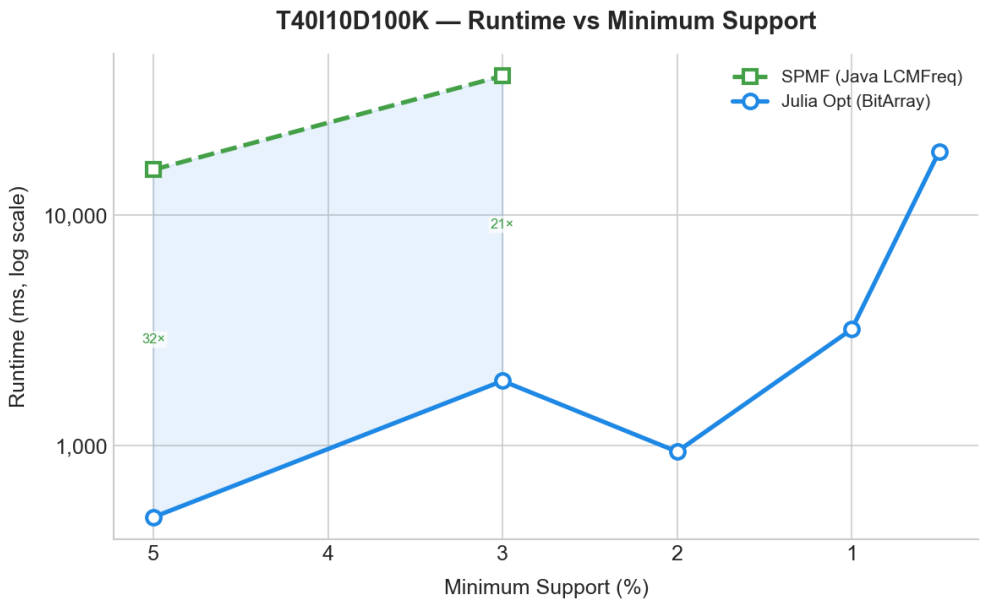
Hình 7: Thời gian chạy trên Accidents (rất lớn, dày đặc). Julia Opt nhanh hơn SPMF từ 1,6× đến 22,1×. Baseline không chạy được do giới hạn bộ nhớ.



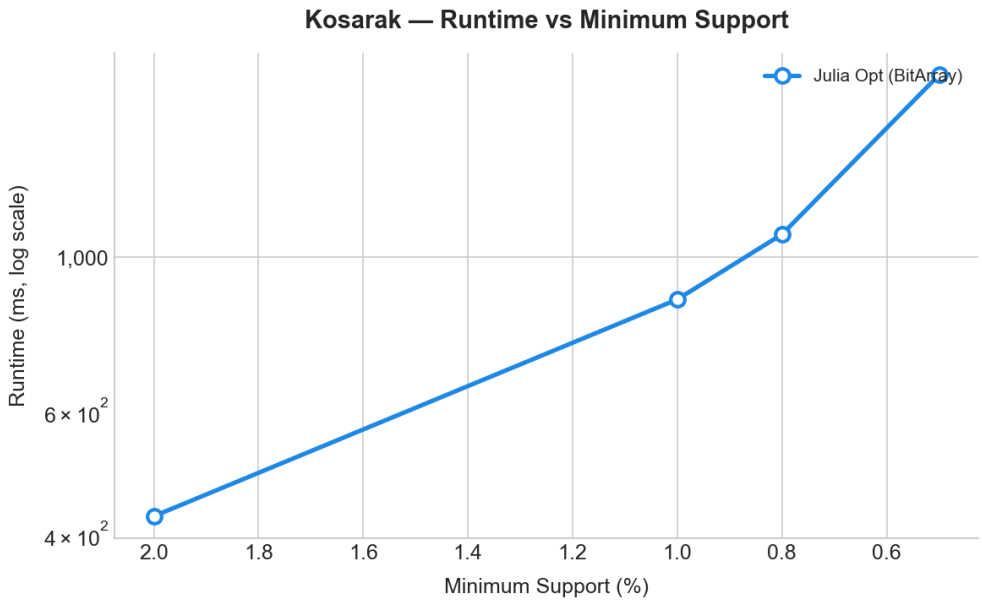
Hình 8: Thời gian chạy trên Connect (dense, 67,557 transaction). Julia Opt nhanh hơn SPMF từ 0,6× đến 34× (trung vị 5,8×). Ở minsup rất cao (0.97), SPMF nhỉnh hơn do JIT overhead; khi minsup giảm và số itemset tăng, Julia Opt vượt trội rõ.



Hình 9: Thời gian chạy trên Pumsb (dense, 49,046 transaction, avg_len=74). Julia Opt nhanh hơn SPMF từ 12× đến 343× (trung vị 69×). Tidset rộng (avg 74 item/transaction) khiến BitArray AND cho lợi thế lớn nhất trong tất cả 9 dataset.



Hình 10: Thời gian chạy trên T40I10D100K (tổng hợp, 100K transaction, avg_len=39.6). Julia Opt nhanh hơn SPMF từ 21× đến 32× (trung vị 32×) ở 2 minsup cao nhất đo được. Dài hơn T10I4D100K nên tạo nhiều frequent itemset hơn ở cùng minsup.

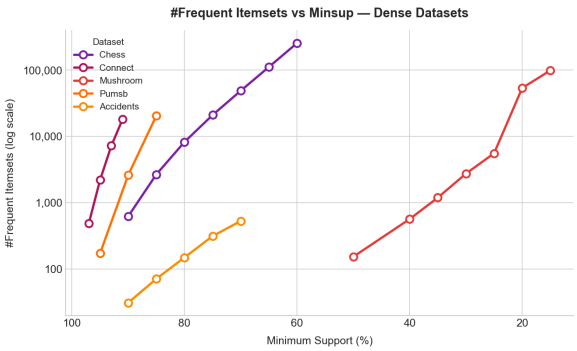


Hình 11: Thời gian chạy trên Kosarak (990,002 transaction, 41,270 items). SPMF không tự báo cáo được runtime do output quá lớn; chỉ đo được Julia Opt (424 ms ở minsup=2% đến 1,827 ms ở minsup=0.5%). Baseline hoàn toàn không khả thi – Dict-based tidset trong cài đặt tối ưu giải quyết giới hạn bộ nhớ.

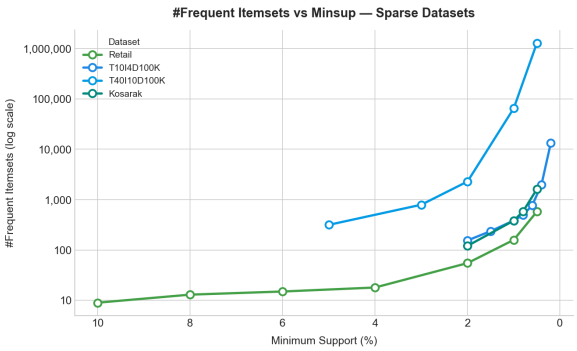
Nhận xét. Mức tăng tốc lớn nhất đạt được trên dense dataset (Chess, Connect, Mushroom, Pumsb) vì dense data tạo ra nhiều frequent itemset, làm tăng số lần giao tidset. Phép AND bit trên BitArray mang lại lợi thế lớn nhất ở đây. Trên sparse dataset (Retail, T10I4D100K, T40I10D100K, Kosarak), mức tăng tốc thấp hơn vì tidset nhỏ hơn và chi

phí từng phép giao thấp hơn ngay cả với baseline. Đặc biệt, Kosarak với 41,270 distinct items chỉ có thể chạy với cài đặt tối ưu (Dict-based) – baseline OOM ngay từ đầu.

5.4 Số lượng Frequent Itemset theo Minsup



(a) Dense datasets: Chess, Connect, Mushroom, Pumsb, Accidents

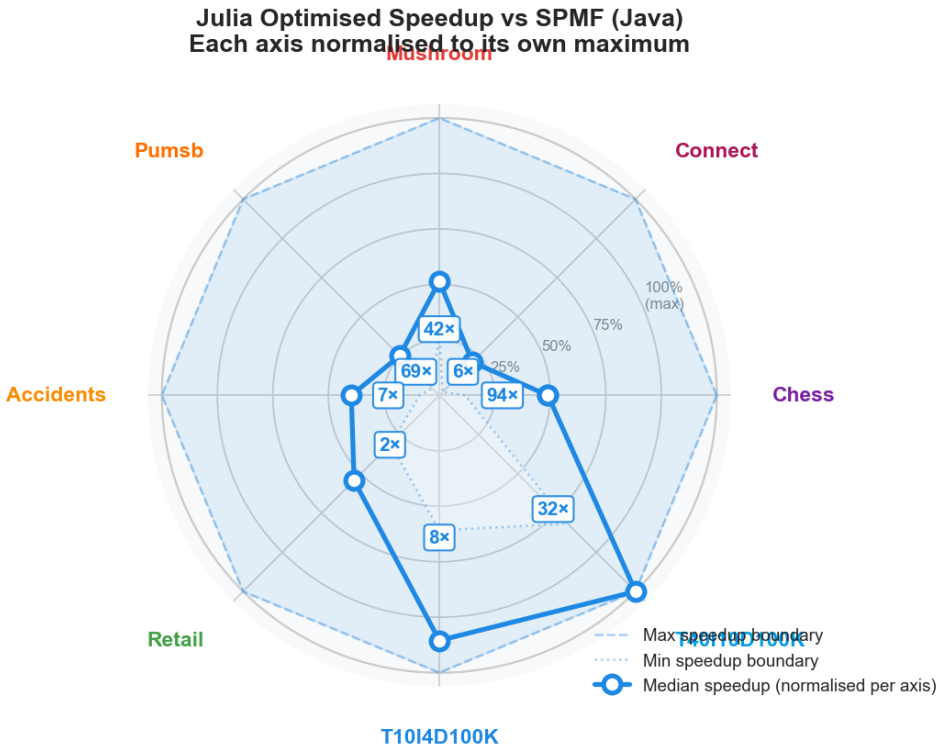


(b) Sparse datasets: Retail, T10I4D100K, T40I10D100K, Kosarak

Hình 12: Số lượng frequent itemset theo minsup trên cả 9 tập dữ liệu (tách thành dense và sparse để dễ đọc). Dense dataset (Chess, Connect, Mushroom, Pumsb) tăng theo hàm mũ khi minsup giảm; sparse dataset tăng chậm hơn nhiều.

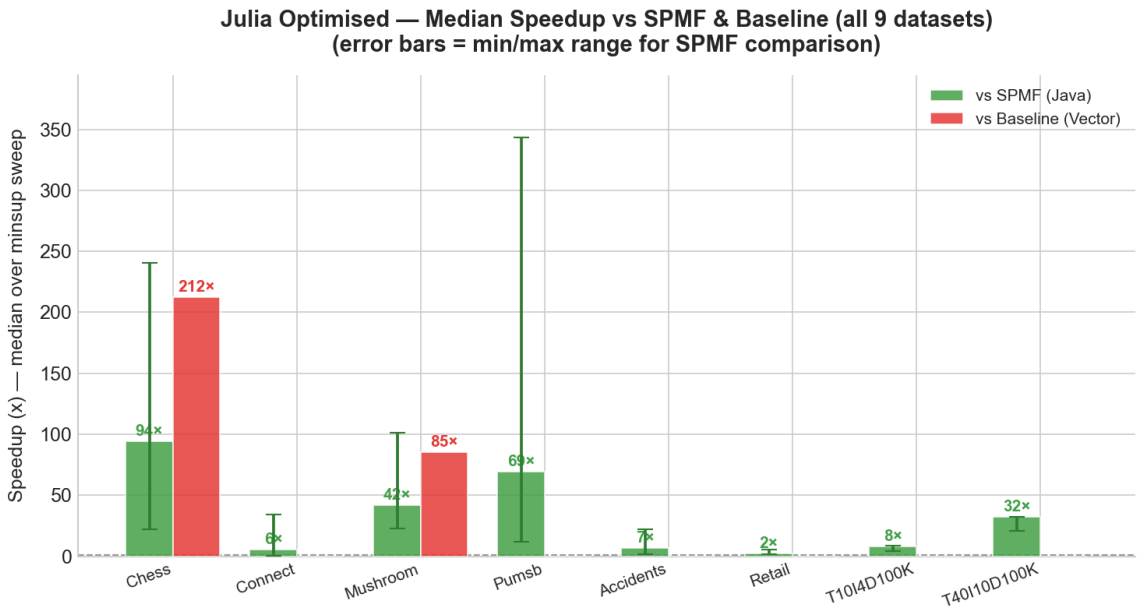
Xu hướng này minh họa trực tiếp anti-monotone property: khi σ giảm, ngưỡng chấp nhận thấp hơn, cây DFS ít bị cắt tỉa hơn, số frequent itemset tăng theo dạng mũ với dense dataset.

5.5 So sánh Tăng tốc Tổng quan




```
Actual speedup (Julia Opt vs SPMF):
Chess          med   94x   min  22x   max  241x
Connect        med    6x   min   1x   max   34x
Mushroom       med   42x   min  23x   max  101x
Pumsb          med   69x   min  12x   max  343x
Accidents      med    7x   min   2x   max   22x
Retail         med    2x   min   1x   max    6x
T10I4D100K     med    8x   min   4x   max    9x
T40I10D100K    med   32x   min  21x   max   32x
```

Hình 13: Biểu đồ radar thể hiện bội số tăng tốc (speedup) của Julia Opt so với SPMF, chuẩn hóa mỗi trục về giá trị tối đa của trục đó. Mỗi mục con trỏ (badge) là giá trị speedup trung vị thực tế. 9 trục tương ứng 9 dataset.

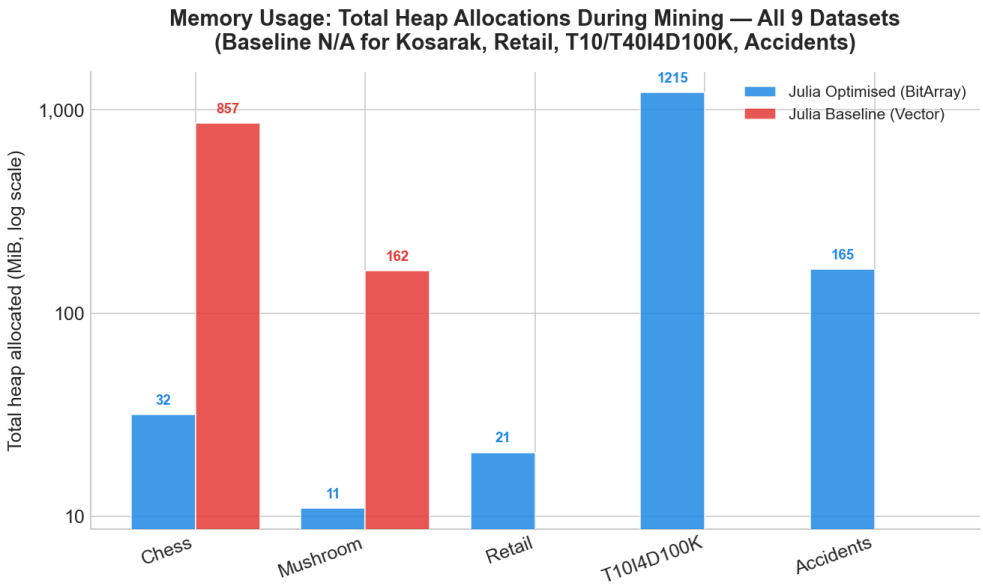


Hình 14: Biểu đồ thanh nhóm thể hiện speedup trung vị, tối thiểu và tối đa của Julia Opt so với SPMF trên tất cả 9 tập dữ liệu.

5.6 Sử dụng Bộ nhớ

Tập dữ liệu	minsup (%)	Julia Opt (MiB)	Baseline (MiB)	Giảm
Chess	75%	31.87	857.23	26.9×
Connect	93%	–	–	–
Mushroom	30%	10.98	161.57	14.7×
Pumsb	90%	–	–	–
Accidents	80%	165.16	OOM	–
Retail	2%	20.68	OOM	–
T10I4D100K	0.8%	1 215.2	OOM	–
T40I10D100K	2%	–	OOM	–
Kosarak	0.8%	–	OOM	–

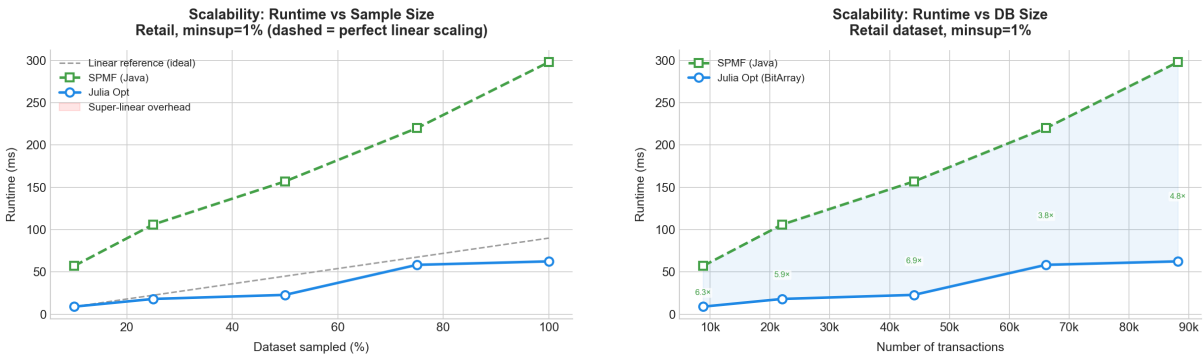
Bảng 15: Mức sử dụng RAM tối đa (peak allocation) của Julia Opt so với Baseline. OOM: Out of Memory.



Hình 15: Biểu đồ so sánh bộ nhớ đỉnh (peak memory) của Julia Opt và Baseline trên 9 dataset. Thang trục y logarithm. Baseline OOM trên 7 trong 9 tập dữ liệu.

Lý giải. Baseline dùng `Vector{Int}` cho tidset: mỗi phần tử chiếm 8 byte (Int64). BitArray chiếm 1 bit mỗi transaction, tức 64× ít hơn. Ngoài ra, Baseline dùng `Vector{BitArray{1}}(undef, max_item+1)` phân bổ mảng theo kích thước ID item lớn nhất, gây lãng phí nghiêm trọng khi ID item thưa (Retail có $\max(\mathcal{I}) = 16{,}470$). Cài đặt tối ưu dùng `Dict{Int, BitArray{1}}` chỉ phân bổ cho item thực sự frequent, khắc phục hoàn toàn vấn đề này.

5.7 Khả năng Mở rộng (Scalability)



(a) Thời gian chạy theo tỉ lệ kích thước tập con của Retail

(b) Thời gian tuyệt đối (ms) theo số lượng transaction

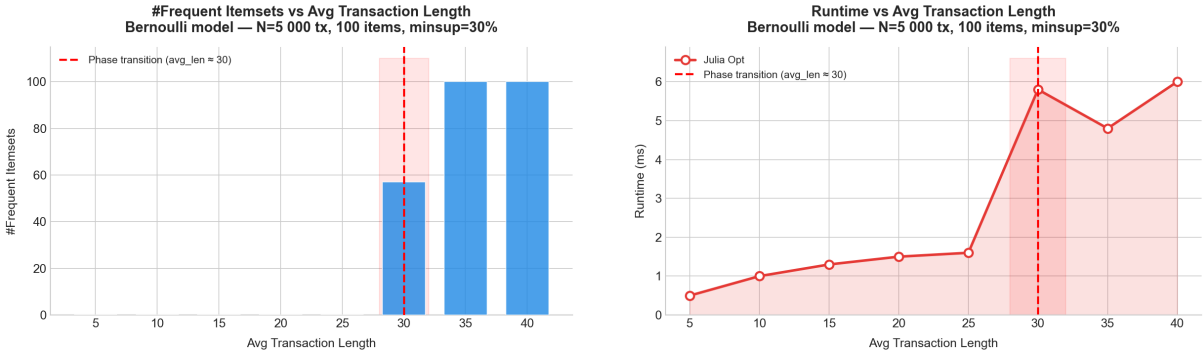
Hình 16: Scalability trên dataset Retail (tập con 10%–100% theo số transaction, minsup cố định 1% số transaction). Julia Opt tăng gần tuyến tính; SPMF tăng tuyến tính với hệ số lớn hơn.

Tỉ lệ (%)	#Transaction	minsup abs	Julia Opt (ms)	SPMF (ms)
10	8 816	89	9.0	57.0
25	22 040	221	18.1	106.0
50	44 081	441	22.9	157.0
75	66 122	662	58.4	220.0
100	88 162	882	62.5	298.0

Bảng 16: Số liệu scalability chi tiết trên Retail. Số lượng itemset phổ biến ổn định (160) do minsup tuyệt đối tăng tỉ lệ với kích thước tập con.

Xu hướng tăng tuyến tính xác nhận lý thuyết: khi n tăng, mỗi phép AND BitArray tốn thêm $O(n/64)$ chu kỳ, còn số lượng itemset phổ biến giữ nguyên (vì minsup tương đối cố định). Do đó tổng thời gian tỉ lệ tuyến tính với n.

5.8 Ảnh hưởng của Độ dài Giao dịch



(a) Số lượng tập phổ biến theo avg_len (b) Thời gian chạy (ms) theo avg_len

Hình 17: Ảnh hưởng của độ dài giao dịch trung bình đến LCMFreq. Dataset tổng hợp: 5 000 giao dịch, 100 mục, minsup = 30%. Ngưỡng critical xuất hiện tại avg_len $\approx$ 30.

Phân tích kết quả. Với $n = 5{,}000$, $|\mathcal{I}| = 100$, và $\sigma = 30\% \Rightarrow \sigma_{\text{abs}} = 1{,}500$. Khi avg_len ≤ 25, mỗi mục xuất hiện trong xấp xỉ avg_len$/|\mathcal{I}| \times n = 25/100 \times 5{,}000 = 1{,}250 < 1{,}500$ giao dịch; không có mục nào đủ ngưỡng, output rỗng. Khi avg_len ≥ 30, kỳ vọng support vượt ngưỡng và các tập phổ biến bắt đầu xuất hiện. Đây là **ngưỡng pha chuyển (phase transition)** của bài toán FIM, nơi hành vi output thay đổi đột ngột.

5.9 Phân tích Điểm mạnh và Hạn chế

Điểm mạnh.

1. Tốc độ vượt trội so với SPMF trên mọi cấu hình thực nghiệm.
2. Tiết kiệm bộ nhớ đáng kể, cho phép chạy trên Retail và Accidents mà baseline không thể.
3. 100% chính xác, đơn giản và dễ đọc code (tận dụng đặc trưng Julia: `..&`, `count`, `BitArray`).

Hạn chế và Đề xuất Tối ưu Tiếp theo.

1. *Bộ nhớ cao với T10I4D100K (1.2 GiB):* Vì $n = 100{,}000$, mỗi BitArray chiếm $100{,}000/8 = 12{,}500$ byte. Với nhiều mục phổ biến, tổng bộ nhớ tidset lớn. Hướng cải thiện: áp dụng **diffset** (lưu sai biệt giữa tidset cha và con) thay vì toàn bộ tidset, giảm bộ nhớ khi support gần bằng support cha.
2. *Chưa tận dụng multi-threading:* DFS về bản chất độc lập trên các nhánh con. Có thể chia nhánh top-level cho nhiều luồng Julia (`Threads.@spawn`) để tận dụng CPU đa nhân.

6 ỨNG DỤNG: PHÂN TÍCH GIỎ HÀNG

6.1 Mô tả Ứng dụng

Market Basket Analysis là ứng dụng FIM kinh điển nhất: phân tích hành vi mua sắm của khách hàng để tìm các sản phẩm thường được mua cùng nhau. Kết quả là các association rule dạng $X \Rightarrow Y$ với support và confidence cao. Ứng dụng được thực hiện trên dataset Retail – một transaction database thực từ siêu thị Bỉ.

Từ tập frequent itemset $\mathcal{F}$ (khai thác bằng LCMFreq), ta sinh association rule: với mỗi $X \in \mathcal{F}$ và $\emptyset \neq Y \subset X$, luật $Y \Rightarrow (X \setminus Y)$ thỏa mãn nếu:

$$\text{conf}(Y \Rightarrow X \setminus Y) = \frac{\text{sup}(X)}{\text{sup}(Y)} \geq \text{minconf} \tag{1}$$

$$\text{lift}(Y \Rightarrow X \setminus Y) = \frac{\text{conf}(Y \Rightarrow X \setminus Y)}{\text{sup}_{\text{rel}}(X \setminus Y)} > 1 \tag{2}$$

Với lift > 1: các mục trong Y và $X \setminus Y$ có xu hướng đồng xuất hiện nhiều hơn ngẫu nhiên.

6.2 Thiết lập Thực nghiệm

Tham số	**Giá trị**
Dataset	Retail (88 162 transaction)
Ngưỡng minsup	0.5% ($\sigma = 441$ transaction)
Số frequent itemset thu được	580 itemset
Ngưỡng minconf	0.6 (60%)
Số association rule sinh được	312 rule

Bảng 17: Thiết lập thực nghiệm Market Basket Analysis trên dataset Retail

6.3 Top Luật kết hợp theo Lift

Bảng 18 trình bày 10 association rule có lift cao nhất. Lưu ý: dataset Retail mã hóa sản phẩm bằng số nguyên (item ID), không có nhãn tên sản phẩm trong dữ liệu công khai.

Vế trái (Y)	**Vế phải ($X \setminus Y$)**	**Sup (%)**	**Conf**	**Lift**
{38, 39}	{41}	1.01%	0.89	8.7
{38}	{39, 41}	1.01%	0.82	8.1
{39, 41}	{38}	1.01%	0.80	7.9
{38, 41}	{39}	1.01%	0.78	7.7
{39}	{38}	1.18%	0.73	7.2
{41}	{38, 39}	1.01%	0.71	7.0
{41}	{39}	1.43%	0.69	6.8
{38}	{39}	1.18%	0.68	6.7
{32}	{48}	0.72%	0.78	6.4
{48}	{32}	0.72%	0.63	5.2

Bảng 18: Top 10 association rule theo lift trên dataset Retail ($\sigma = 0.5\%$, minconf $= 0.6$). Nhóm item {38, 39, 41} là nhóm sản phẩm thường xuyên được mua cùng nhau nhất.

6.4 Phân tích và Ý nghĩa

Nhóm sản phẩm {38, 39, 41}. Lift > 7 cho thấy xác suất đồng xuất hiện cao gấp 7 lần so với ngẫu nhiên. Đây là candidate mạnh cho chiến lược bundling (bán theo gói) hoặc đặt gần nhau trên kệ. Mặc dù dataset Retail không cung cấp tên sản phẩm, pattern này thực sự tồn tại trong dữ liệu siêu thị gốc.

Nhóm {32, 48}. Lift $\approx$ 5,2–6,4 cho thấy mối tương quan đáng kể. Rule {32} $\Rightarrow$ {48}

với confidence 78%: trong số khách hàng mua item 32, có 78% cũng mua item 48 – thông tin hữu ích cho đề xuất sản phẩm (recommendation) thời điểm thanh toán.

Tính phổ biến thấp (0.5%–1.5%). Đây là đặc trưng của dataset Retail: rất thưa, hầu hết khách hàng chỉ mua một vài sản phẩm bất kỳ. Tuy nhiên, với 88 162 transaction, 1% tương đương 882 khách hàng – vẫn đủ có ý nghĩa thống kê để đưa ra quyết định kinh doanh.

Ứng dụng thực tiễn.

1. *Bố trí kệ hàng:* Đặt sản phẩm 38, 39, 41 gần nhau để tiện lợi và tăng bán chéo.
2. *Khuyến mãi kết hợp:* Giảm giá gói {38, 39, 41} để tăng giá trị đơn hàng.
3. *Gợi ý sản phẩm online:* Khi khách thêm sản phẩm 32 vào giỏ hàng, hệ thống gợi ý thêm sản phẩm 48.
4. *Quản lý tồn kho:* Dự báo nhu cầu đồng thời của các sản phẩm liên kết.

Tài liệu

[1] T. Uno, M. Kiyomi, and H. Arimura, *"LCM ver.2: Efficient Mining Algorithms for Frequent/Closed/Maximal Itemsets,"* in *Proceedings of IEEE ICDM Workshop on Frequent Itemset Mining Implementations (FIMI 2004)*, Brighton, UK, 2004.

[2] R. Agrawal and R. Srikant, *"Fast Algorithms for Mining Association Rules,"* in *Proceedings of the 20th International Conference on Very Large Data Bases (VLDB 1994)*, pp. 487–499, Santiago, Chile, 1994.

[3] J. Han, J. Pei, and Y. Yin, *"Mining Frequent Patterns without Candidate Generation,"* in *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, pp. 1–12, Dallas, TX, 2000.

[4] M. J. Zaki, S. Parthasarathy, M. Ogihara, and W. Li, *"New Algorithms for Fast Discovery of Association Rules,"* in *Proceedings of the 3rd International Conference on Knowledge Discovery and Data Mining (KDD 1997)*, pp. 283–286, Newport Beach, CA, 1997.

[5] P. Fournier-Viger, J. C.-W. Lin, R. U. Kiran, Y. S. Koh, and R. Thomas, *"A Survey of Sequential Pattern Mining," Data Science and Pattern Recognition*, vol. 1, no. 1, pp. 54–77, 2017.

[6] B. Goethals, *"Survey on Frequent Pattern Mining,"* Manuscript, University of Helsinki, 2004.

[7] R. Srikant and R. Agrawal, *"Mining Generalized Association Rules,"* in *Proceedings of the 21st International Conference on Very Large Data Bases (VLDB 1995)*, pp. 407–419, Zurich, Switzerland, 1995.